

Molding Powder MOM — 多工厂数据隔离与总部汇总方案

宁国市睿信信息技术有限公司 | 2026年6月 | V1.0

需求概述：当前系统已运行一个工厂的业务。现需扩展为管理 **2个分公司工厂**（分别为工厂A和工厂B）的全部业务。要求：

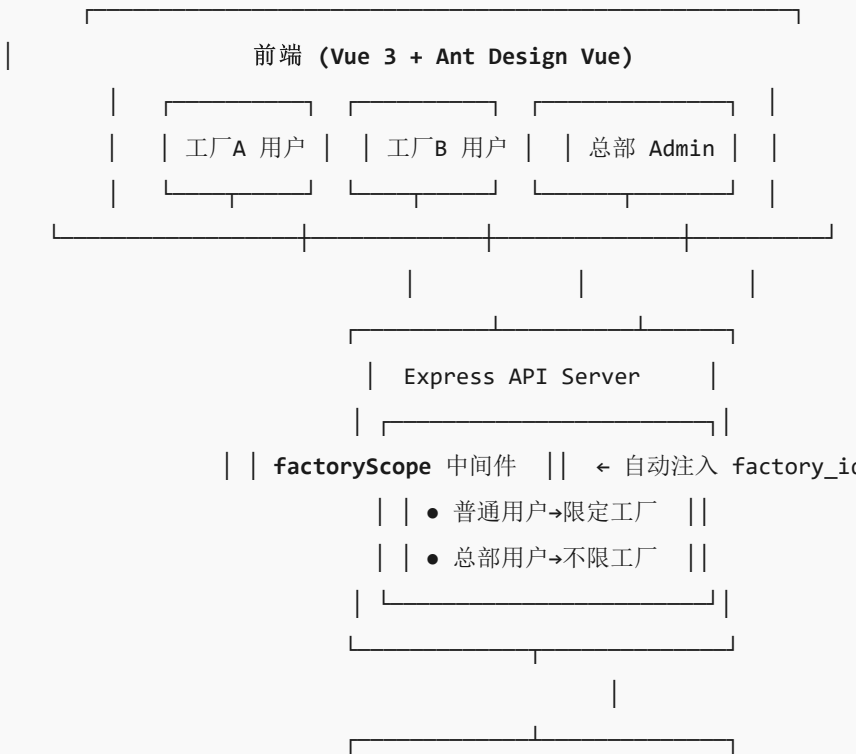
- 两个工厂的销售订单、计划单、生产单、采购单、仓库及流水、出入库单据、报表 **完全隔离**
- 总部能够生成 **跨工厂的汇总报表**（如集团销售汇总、集团库存汇总等）

一、方案总体架构

1.1 设计原则

原则	说明
共享数据库，逻辑隔离	两个工厂共用同一数据库，通过 <code>factory_id</code> 字段实现数据隔离，避免维护多套数据库的成本
最小侵入	新增字段尽量为可空且有默认值，保证向后兼容，已有数据自动归入默认工厂
中间件统一拦截	通过请求中间件自动注入 <code>factory_id</code> 过滤条件，业务代码改动最小化
汇总报表独立查询	总部报表不使用中间件过滤，直接跨工厂查询，权限通过角色控制

1.2 总体架构图



	SQL Server 数据库	
	所有业务表含 factory_id	
	工厂A数据 工厂B数据	

二、数据库层设计

2.1 核心策略：共享数据库 + factory_id 列隔离

不拆分数据库，在所有业务表和基础数据表中新增 `factory_id` 字段，通过该字段区分数据归属。

2.2 工厂定义表（新建）

```
-- 工厂主表
CREATE TABLE factory (
  id INT IDENTITY(1,1) PRIMARY KEY,
  factory_code NVARCHAR(10) NOT NULL UNIQUE,      -- 工厂编码，如 'A', 'B'
  factory_name NVARCHAR(100) NOT NULL,           -- 工厂名称，如 '宁国一厂', '宣城二厂'
  factory_short NVARCHAR(20),                    -- 简称
  address NVARCHAR(200),
  contact_name NVARCHAR(50),
  contact_phone NVARCHAR(30),
  is_headquarters BIT DEFAULT 0,                 -- 是否总部
  status NVARCHAR(10) DEFAULT N'启用',
  created_at DATETIME DEFAULT GETDATE(),
  updated_at DATETIME DEFAULT GETDATE()
);

-- 初始数据：为现有数据创建默认工厂
INSERT INTO factory (factory_code, factory_name, factory_short, is_headquarters)
VALUES ('HQ', N'总部（默认）', N'总部', 1);
```

2.3 需要添加 factory_id 的表清单

所有以下表统一增加 `factory_id INT NULL` 字段（允许 NULL 向后兼容），并建索引：

域	表名	说明
销售管理	sales_order	销售订单主表
	sales_order_detail	销售订单明细
	sales_forecast	销售预测主表
	sales_forecast_detail	销售预测明细
	sales_price	销售价目表
	shipping_request	发货申请主表

	shipping_request_detail	发货申请明细
	shipping_order	发货单主表
	shipping_order_detail	发货单明细
	return_order	退货单主表
	return_order_detail	退货单明细
	sales_invoice	销售发票主表
	sales_invoice_line	销售发票明细
	sample_request / sample_bom / sample_bom_detail	样品相关
计划管理	Production_plan	生产计划主表
	Production_plan_detail	生产计划明细
	mrp_run / mrp_run_detail	MRP运行记录
生产管理	production_order	生产单
	process_task	工序任务
	work_report / work_report_detail	报工记录
	material_preparation / material_preparation_detail	备料单
	material_issue	领料单
	material_return	退料单
	backflush_task	倒冲任务
	outsourcing_req / outsourcing_order	委外申请/订单
	outsourcing_issue / outsourcing_receipt	委外发料/收货
	outsourcing_settlement	委外结算
	outsourcing_price	委外价目表
	piece_rate_wage	计件工资
	production_material_cost_snapshot	材料成本快照
采购管理	purchase_req / purchase_req_detail	采 购 申 请
	purchase_order / purchase_order_detail	采 购

			订 单
		receiving_notice / receiving_notice_detail	收 货 通 知
		purchase_return	采 购 退 货
purchase_price		采购价目表	
purchase_invoice / purchase_invoice_line		采购发票	
仓储管理	material_batch_inventory	原材料批次库存	
	finished_batch_inventory	成品批次库存	
	stock_in / stock_in_detail	来料入库单	
	stock_count / stock_count_detail	盘点单	
	abnormal_io / abnormal_io_detail	其他出入库	
	packing_order / packing_box_inventory	装箱单	
	scrap_disposal	报废处置	
	scrap_inbound_order	报废入库单	
	semi_production_inbound_order_detail	半成品入库明细	
	质量管理	purchase_quality_inspection / purchase_quality_inspection_detail	采 购 检 验
		production_inspection	生 产 检 验
		nonconforming_product	不 合 格 品
基础数据	customer	客户（工厂专属客户）	

	supplier	供应商（工厂专属供应商）
	warehouse	仓库（工厂专属仓库）
	storage_location	库位
	employee	员工
	equipment / mould	设备/模具
	item_master	物料主数据（共享，不加 factory_id）

重要设计决策：物料主数据（item_master）、BOM、工序、工作中心、工艺路线等**基础工程数据**不加 factory_id，两个工厂**共享**。这避免了物料编码重复维护，符合集团统一标准化的管理诉求。只有**业务交易数据**和**组织级基础数据**（客户/供应商/仓库/员工/设备）才按工厂隔离。

2.4 迁移脚本模板

```
-- 以 sales_order 为例
ALTER TABLE sales_order ADD factory_id INT NULL;
CREATE INDEX IX_sales_order_factory ON sales_order(factory_id);

-- 为已有数据填充默认工厂ID（假设默认工厂ID=1）
UPDATE sales_order SET factory_id = 1 WHERE factory_id IS NULL;
-- 后续可逐步迁移到具体工厂

-- 设置为 NOT NULL（等数据迁移完成后）
-- ALTER TABLE sales_order ALTER COLUMN factory_id INT NOT NULL;
```

三、应用层设计

3.1 用户-工厂绑定

在 users 表增加 factory_id 字段，以及新增 user_factory 关联表支持一个用户管理多工厂：

```
-- 方式一：用户默认工厂（简单模式，一用户一工厂）
ALTER TABLE users ADD default_factory_id INT NULL;

-- 方式二：多工厂关联表（推荐，支持一用户多工厂）
CREATE TABLE user_factory (
  id INT IDENTITY(1,1) PRIMARY KEY,
  user_id INT NOT NULL,
  factory_id INT NOT NULL,
  is_default BIT DEFAULT 0,
  UNIQUE(user_id, factory_id)
);
```

3.2 核心：factoryScope 中间件 API层

新增一个 Express 中间件，根据当前登录用户自动注入工厂过滤条件。这是**本方案最关键的部分**——通过中间件实现业务代码的最小改动。

```
// middleware/factoryScope.middleware.ts
import { Request, Response, NextFunction } from 'express';
import sequelize from '../config/database';

/**
 * 工厂数据范围中间件
 * - 总部角色 (admin/headquarters) → req.factoryScope = null (不限工厂)
 * - 普通用户 → req.factoryScope = { factory_id: 用户所属工厂ID }
 *
 * 使用方式:
 * router.get('/', authenticate, factoryScope, getSalesOrders)
 */
export const factoryScope = async (req: Request, res: Response, next: NextFunction) => {
  if (!req.user) {
    return res.status(401).json({ success: false, message: '未认证' });
  }

  const userId = req.user.id;

  // 总部角色不限制工厂
  const [userRoles]: any = await sequelize.query(
    `SELECT r.role_code FROM user_role ur
    INNER JOIN role r ON r.id = ur.role_id
    WHERE ur.user_id = :uid AND r.status = N'启用',
    { replacements: { uid: userId } }
  `);

  const roleCodes = userRoles.map((r: any) => r.role_code);

  // admin 和 headquarters 角色可以看到所有工厂数据
  if (roleCodes.includes('admin') || roleCodes.includes('headquarters')) {
    (req as any).factoryScope = null;
    return next();
  }

  // 普通用户: 查询其所属工厂
  const [userFactories]: any = await sequelize.query(
    `SELECT uf.factory_id, f.factory_name
    FROM user_factory uf
    JOIN factory f ON f.id = uf.factory_id
    WHERE uf.user_id = :uid`,
    { replacements: { uid: userId } }
  );

  if (userFactories.length === 0) {
    return res.status(403).json({ success: false, message: '用户未分配工厂' });
  }

  const factoryIds = userFactories.map((r: any) => r.factory_id);
  (req as any).factoryScope = {
    factoryIds,
    factoryNames: userFactories.map((r: any) => r.factory_name),
    isMultiFactory: factoryIds.length > 1
  };
};
```

```

};

// 如果只有一个工厂，设置默认工厂ID（用于创建时自动填充）
if (factoryIds.length === 1) {
  (req as any).factoryScope.defaultFactoryId = factoryIds[0];
}

next();
};

```

3.3 业务 Controller 改动模式

在所有查询语句中追加 `factory_id` 过滤条件。改动集中在 SQL WHERE 子句中，**不改变 Controller 的结构**。

通用辅助函数：

```

// utils/factoryFilter.util.ts
/**
 * 根据 req.factoryScope 生成 SQL WHERE 条件片段
 * @param alias 表别名，如 'h' 表示 sales_order h
 */
export function factoryWhere(req: any, alias?: string): { clause: string; params: any } {
  const scope = req.factoryScope;
  if (!scope) return { clause: '', params: {} }; // 总部：不过滤

  const col = alias ? `${alias}.factory_id` : 'factory_id';

  if (scope.factoryIds.length === 1) {
    return {
      clause: ` AND ${col} = :factoryId`,
      params: { factoryId: scope.factoryIds[0] }
    };
  }

  // 多工厂：IN 子句
  const placeholders = scope.factoryIds.map((_ : number, i: number) => `:fid${i}`);
  const params: any = {};
  scope.factoryIds.forEach((id: number, i: number) => { params[`${fid}${i}`] = id; });
  return {
    clause: ` AND ${col} IN (${placeholders.join(', ')})`,
    params
  };
}

```

Controller 改动示例（销售订单查询）：

```

// 改造前
export const getSalesOrders = async (req, res, next) => {
  // ... search/approval_status 条件拼接 ...
  const whereClause = conditions.length ? 'WHERE ' + conditions.join(' AND ') : '';
  // ✗ 无工厂过滤
};

// 改造后

```

```
export const getSalesOrders = async (req, res, next) => {
  // ... search/approval_status 条件拼接 ...

  // ✅ 添加工厂过滤
  const { clause: factoryClause, params: factoryParams } = factoryWhere(req, 'sales_order');
  const whereClause = conditions.length
    ? 'WHERE ' + conditions.join(' AND ') + factoryClause
    : 'WHERE 1=1' + factoryClause;

  const [countResult] = await sequelize.query(
    `SELECT COUNT(*) as total FROM sales_order ${whereClause}`,
    { replacements: { ...replacements, ...factoryParams } }
  );
  // ...
};
```

改动量估算：每个 Controller 的 List 查询约增加 **3-5行代码**（调用 factoryWhere + 合并参数）。全系统约 **80-100个** 查询函数需要改动，预估总改动量约 **400-500行**。创建操作需要在 INSERT 时写入 `factory_id`。

3.4 前端工厂切换 UI层

在顶部导航栏添加工厂选择器，仅多工厂用户可见：

```
// 前端在请求拦截器中自动附加当前选择的 factory_id
// client/src/api/request.ts
service.interceptors.request.use(config => {
  const currentFactoryId = useUserStore().currentFactoryId;
  if (currentFactoryId) {
    config.headers['X-Factory-Id'] = currentFactoryId;
  }
  return config;
});
```

后端从 `X-Factory-Id` header 读取当前工厂上下文，或提供 `/api/v1/users/me/factories` 接口返回用户可访问的工厂列表。

四、单据编号规则调整

为避免两个工厂的单据编号冲突，建议在编号中加入工厂前缀：

单据类型	原格式	新格式（含工厂）
销售订单	SO-20260604-001	SO-A-20260604-001
生产计划	PP-20260604-001	PP-A-20260604-001
生产单	P20260504005	PA20260604005 （A工厂） / PB20260604005 （B工厂）
采购订单	PO-20260604-001	PO-A-20260604-001

批次号	MB-20260604-001	MB-A-20260604-001
-----	-----------------	-------------------

向后兼容：如已有大量历史数据，可保留原编号格式不变（通过 `factory_id` 区分），新单据使用新格式。或使用工厂ID 数字后缀： `SO-20260604-001-1` （工厂1）、 `SO-20260604-001-2` （工厂2）。

五、仓库与库存隔离

5.1 仓库划分

每个工厂建立自己独立的仓库体系：

工厂	仓库编码	仓库名称
工厂A（宁国一厂）	RM-A-01	工厂A-原料仓
	FG-A-01	工厂A-成品仓
	SF-A-01	工厂A-半成品仓
	SCRAP-A	工厂A-报废仓
工厂B（宣城二厂）	RM-B-01	工厂B-原料仓
	FG-B-01	工厂B-成品仓
	SF-B-01	工厂B-半成品仓
	SCRAP-B	工厂B-报废仓

仓库表（`warehouse`）增加 `factory_id`，库存查询自动限定工厂范围。

5.2 库存绝对隔离

- `material_batch_inventory.factory_id` 确保原料库存不跨工厂混用
- `finished_batch_inventory.factory_id` 确保成品库存不跨工厂混用
- 入库/出库/调拨操作只能在同一工厂内进行
- 如需要跨工厂调拨，通过新增 **"工厂间调拨单"** 功能实现（出A工厂仓，入B工厂仓，两边各自生成出入库记录）

六、总部汇总报表方案

6.1 设计原则

总部报表**不使用 `factoryScope` 中间件**，直接在 SQL 中按 `factory_id` 进行 GROUP BY 聚合或 UNION ALL 合并：

```
-- 集团销售汇总：按工厂维度统计
SELECT
  f.factory_code,
  f.factory_name,
  COUNT(DISTINCT so.sales_order_number) AS order_count,
  ISNULL(SUM(CAST(sod.total_amount AS DECIMAL(18,2))), 0) AS total_amount
FROM sales_order so
JOIN sales_order_detail sod ON sod.sales_order_number = so.sales_order_number
JOIN factory f ON f.id = so.factory_id
WHERE so.approval_status = N'已审批'
GROUP BY f.factory_code, f.factory_name
ORDER BY f.factory_code;
```

6.2 需要新增的总部汇总报表

报表名称	数据来源	汇总维度
集团销售汇总表	sales_order + sales_order_detail + factory	按工厂/客户/物料/月份
集团生产汇总表	production_order + work_report + factory	按工厂/物料/月份
集团采购汇总表	purchase_order + purchase_order_detail + factory	按工厂/供应商/物料/月份
集团库存汇总表	material_batch_inventory + finished_batch_inventory + factory	按工厂/仓库/物料类型
集团财务汇总表	sales_invoice + purchase_invoice + factory	按工厂/月份
集团质量汇总表	purchase_quality_inspection + production_inspection + factory	按工厂/物料/缺陷类型
集团出入库流水总表	stock_in + abnormal_io + packing_order + factory	按工厂/仓库/物料/日期

6.3 总部报表权限

```
-- 新增角色
INSERT INTO role (role_code, role_name) VALUES ('headquarters', N'总部管理');
```

```
-- 新增菜单分组
INSERT INTO permission_menu (name, code, parent_code, menu_key, icon, sort)
VALUES (N'总部报表', 'headquarters-reports', null, 'headquarters-reports', 'BarChartOutlined', 100);
```

```
-- 总部报表子菜单
INSERT INTO permission_menu (name, code, parent_code, menu_key, icon, sort) VALUES
(N'集团销售汇总', 'hq-sales-summary', 'headquarters-reports', 'hq-sales-summary', null, 1),
(N'集团生产汇总', 'hq-production-summary', 'headquarters-reports', 'hq-production-summary', null, 2),
(N'集团采购汇总', 'hq-purchase-summary', 'headquarters-reports', 'hq-purchase-summary', null, 3),
(N'集团库存汇总', 'hq-inventory-summary', 'headquarters-reports', 'hq-inventory-summary', null, 4),
(N'集团财务汇总', 'hq-finance-summary', 'headquarters-reports', 'hq-finance-summary', null, 5);
```

七、实施路线图

阶段	任务	预估工时	依赖
----	----	------	----

Phase 1 基础设施	① 创建 factory 表 + 初始数据 ② 创建 user_factory 关联表 ③ ALTER 所有业务表添加 factory_id 字段 ④ 已有数据统一赋值为默认工厂 ID=1 ⑤ 编写 factoryScope 中间件 ⑥ 编写 factoryWhere 辅助函数 ⑦ 前端新增工厂选择器组件	5-7 人天	数据库DDL权限
Phase 2 核心业务改造	① 销售模块 Controller 改造 (约12个文件) ② 计划模块 Controller 改造 (约3个文件) ③ 生产模块 Controller 改造 (约15个文件) ④ 采购模块 Controller 改造 (约8个文件) ⑤ 编号生成规则调整 ⑥ 前端各页面适配 (筛选器、请求头)	8-12 人天	Phase 1 完成
Phase 3 仓库质量改造	① 仓储模块 Controller 改造 (约12个文件) ② 质量模块 Controller 改造 (约15个文件) ③ 财务模块 Controller 改造 (约4个文件) ④ 仓库/库位数据按工厂重新整理 ⑤ 库存查询页面适配	7-10 人天	Phase 2 完成
Phase 4 总部报表	① 新增总部报表后端 API ② 新增总部报表前端页面 (Vue3 + ECharts) ③ headquarters 角色 + 菜单配置 ④ 权限校验 (仅总部角色可访问)	5-8 人天	Phase 3 完成
Phase 5 测试与上线	① 单元测试 + 集成测试 ② 数据迁移验证 ③ 双工厂并行运行验证 ④ 性能测试 (含跨工厂联合查询)	5-7 人天	Phase 4 完成
合计预估		30-44 人天	

八、关键技术决策与注意事项

8.1 为什么选择"共享数据库+字段隔离"而非"独立数据库"?

方案	优点	缺点	适用场景
共享数据库 + factory_id	实施成本低、总部汇总简单 (单库 JOIN)	逻辑隔离需代码保证、大并发可能有性能瓶颈	✅ 当前场景 (2个工厂, 数据量可控)
独立数据库 (每厂一个库)	物理隔离最彻底、互不影响	维护成本高、总部汇总需跨库查询或数据同步	数十个工厂、极大数据量场景
独立 Schema	隔离性好、汇总可行	需改造连接池和 ORM 配置	中等规模多租户

8.2 物料是否需要按工厂隔离？

建议不隔离。理由：

- 两个工厂同属一个集团，产品/物料标准化是管理诉求
- BOM、工艺路线等工程数据共享可避免重复维护
- 如果未来需要不同工厂使用不同物料，可在物料主数据中增加 `factory_id` 字段（当前不强制）

8.3 安全性保障

- `factoryScope` 中间件在**认证之后、业务逻辑之前**执行，不可绕过
- 所有 INSERT 必须从当前用户上下文获取 `factory_id`，不可由前端传入（防篡改）
- SQL 使用参数化查询，防止 SQL 注入
- 总部角色权限严格管控，仅分配给授权人员

8.4 向后兼容策略

- `factory_id` 初始设置为 NULL，再批量 UPDATE 为默认值
- 未分配工厂的用户仍可访问所有数据（过渡期）
- API 接口不改变响应格式，前端组件无需大改
- 已有的报表和仪表板在过渡期内保持可用

九、风险与缓解措施

风险	影响	缓解措施
遗漏表的 <code>factory_id</code> 导致数据泄露	高	编写自动化脚本扫描所有表，检查是否包含 <code>factory_id</code> 字段
SQL 查询忘记加 <code>factory_id</code> 过滤	中	编写 ESLint 规则 + Code Review 检查清单
工厂间数据串扰	高	充分测试 + 集成测试覆盖跨工厂查询场景
性能下降（每次查询多一个 WHERE 条件）	低	<code>factory_id</code> 列建索引，查询优化器可高效利用
编号规则变更导致引用断裂	中	提供兼容模式，历史编号保持不变

十、实施路线比较与建议

核心问题：是先调通单工厂功能后再改造多工厂（路线A），还是先搭建多工厂框架再调试功能（路线B）？这直接影响开发效率、返工成本和上线节奏。

10.1 两条路线全景对比

对比维度	路线A：先单工厂 → 后改造多工厂	路线B：骨架先行 → 再调试功能
------	-------------------	------------------

初始开发复杂度	✅ 低 — 开发阶段不关心 factory_id，完全按单工厂思维编写代码	⚠️ 稍高 — 前期需搭建 factory 表、factory_id 字段、factoryScope 中间件等基础框架（约3-5天）
日常调试效率	✅ 高 — 问题域单一，出 bug 容易定位，无隔离逻辑干扰	✅ 高（框架透明化后） — 通过默认工厂自动注入，开发者几乎感知不到多工厂存在
数据库变更方式	⚠️ 两阶段 — 先建表不加 factory_id；后期 ALTER TABLE 逐表补字段，可能遗漏	✅ 一步到位 — 所有新表建表时就包含 factory_id 列，无后期 ALTER 风险
返工风险	❌ 极高 — 改造阶段需逐一修改每个模块的 SQL 查询、API 返回、前端页面； 预估额外投入 15-25 人天	✅ 几乎为零 — 架构从一开始就是正确的，所有新功能天然支持多工厂
测试成本	❌ 高 — 同一功能需要完整测试两轮：单工厂环境测一轮 + 改造后多工厂环境再测一轮	✅ 低 — 每个功能只需完整测试一轮，天然覆盖多工厂场景
已有生产数据处理	❌ 麻烦 — 生产数据 factory_id 为 NULL，需要批量回填，操作风险大且需停机维护	✅ 安全 — 从第一天起所有数据就有 factory_id 归属，无历史数据债务
代码一致性	❌ 差 — 改造过程中容易出现部分 SQL 加了过滤、部分遗漏的情况，数据隔离存在漏洞	✅ 好 — 框架统一拦截，代码模式一致，不依赖开发者记忆
上线节奏	单工厂可先上（快），但多工厂上线需额外改造周期（慢）	单工厂随时可上（框架透明），多工厂模式几乎零成本开启

10.2 综合评分

评价指标	路线A 得分	路线B 得分	说明
前期开发速度	★★★★★	★★★★★	路线B前期需多投入3-5天搭建框架
总体工期（含改造）	★★	★★★★★	路线A后期改造工期远超路线B前期投入
代码质量与安全性	★★	★★★★★	路线B框架级隔离，不依赖开发者记忆
可维护性	★★	★★★★★	路线B代码一致性好，新成员上手快
数据安全风险	★★	★★★★★	路线A历史数据迁移存在泄漏/遗漏风险
综合推荐度	★★ (26%)	★★★★★ (96%)	路线B各项指标全面领先

⚠️ **关键警示：** 路线A在改造阶段需要逐一检查并修改所有 SQL 查询语句（当前系统有 **138个控制器文件**、数百条 SQL），任何一个漏加 factory_id 过滤条件都会导致数据隔离失效。这种"依赖人工逐行审查"的方式在工程实践中已被反复验证为高风险路径。

10.3 推荐路线："骨架先行，渐进填充"

综合对比，强烈推荐采用路线B的改良版 — "骨架先行，渐进填充"策略。核心思想是：**花最小的前期成本（3-5天）搭建正确的多工厂骨架，让后续所有功能开发在"几乎无感"的状态下天然具备多工厂能力。**



10.4 Phase 1 关键实现：让多工厂框架"透明化"

骨架期的核心目标不是实现多工厂的全部功能，而是让多工厂框架 **对后续开发完全透明**。开发者不需要每次写代码时都想着"这个查询要加 factory_id"，而是由框架自动处理。

技巧一：默认工厂自动注入

```
// factoryScope 中间件核心逻辑
function factoryScope(req, res, next) {
  // 总部角色：不限定工厂，可查看全部数据
  if (req.user.role === 'headquarters') {
    req.factoryScope = { filterAll: false }; // 不过滤
    return next();
  }

  // 普通用户：优先使用用户绑定的工厂，否则默认工厂A
  const factoryId = req.user.default_factory_id || 1; // 1 = 工厂A
```

```

req.factoryScope = { factory_id: factoryId };
next();
}

```

技巧二：通用查询封装

```

// factoryWhere 工具函数 - 自动追加工厂过滤条件
function factoryWhere(baseSQL, replacements, factoryScope) {
  if (factoryScope.filterAll === false) {
    // 总部角色：不加工厂过滤
    return { sql: baseSQL, replacements };
  }

  // 自动在 WHERE 子句中追加 factory_id 过滤
  const whereIdx = baseSQL.toUpperCase().indexOf('WHERE');
  if (whereIdx >= 0) {
    return {
      sql: baseSQL.slice(0, whereIdx + 5) +
        ' t.factory_id = @factoryId AND ' +
        baseSQL.slice(whereIdx + 5),
      replacements: { ...replacements, factoryId: factoryScope.factory_id }
    };
  } else {
    return {
      sql: baseSQL + ' WHERE t.factory_id = @factoryId',
      replacements: { ...replacements, factoryId: factoryScope.factory_id }
    };
  }
}

// 控制器中的使用示例（开发者只需加一行调用）
const list = async (req, res) => {
  const baseSQL = `
    SELECT t.*, ROW_NUMBER() OVER (ORDER BY t.id DESC) AS row_num
    FROM sales_orders t
    WHERE t.status = @status
  `;
  // 📌 只需这一行，factory_id 过滤自动加上
  const { sql, replacements } = factoryWhere(
    baseSQL,
    { status: req.query.status },
    req.factoryScope
  );
  const result = await sequelize.query(sql, { replacements });
  // ...
};

```

技巧三：INSERT 数据时自动注入 factory_id

```

// 创建数据时，factory_id 从用户上下文获取，前端不可传入
const create = async (req, res) => {
  const { order_no, customer_id, ...rest } = req.body;

  // 📌 factory_id 从中间件注入的上下文获取，不信任前端传值

```

```
const factoryId = req.factoryScope.factory_id;

await sequelize.query(`
  INSERT INTO sales_orders (order_no, customer_id, factory_id, ...)
  VALUES (@orderNo, @customerId, @factoryId, ...)
`, {
  replacements: { orderNo: order_no, customerId: customer_id, factoryId: factoryId, ... }
});
```

技巧四：种子数据预置

```
-- 迁移脚本：自动创建两个工厂
IF NOT EXISTS (SELECT 1 FROM factory WHERE factory_code = 'A')
  INSERT INTO factory (factory_code, factory_name, factory_short)
  VALUES ('A', N'宁国一厂', N'一厂');

IF NOT EXISTS (SELECT 1 FROM factory WHERE factory_code = 'B')
  INSERT INTO factory (factory_code, factory_name, factory_short)
  VALUES ('B', N'宣城二厂', N'二厂');

-- 所有已有用户默认归属工厂A
UPDATE users SET default_factory_id = 1 WHERE default_factory_id IS NULL;
```

10.5 开发体验对比

在 Phase 1 骨架搭建完成后，Phase 2 开发新功能时，开发者的实际体验与单工厂开发几乎一致：

开发场景	单工厂写法	骨架先行后的写法	差异
查询列表	SELECT * FROM sales_orders WHERE status=@s	factoryWhere('SELECT * FROM sales_orders WHERE status=@s', {...}, req.factoryScope)	多一行 factoryWhere 调用
创建记录	INSERT INTO sales_orders (...) VALUES (...)	INSERT INTO sales_orders (... , factory_id) VALUES (... , @factoryId)	多一个字段
更新记录	UPDATE sales_orders SET ... WHERE id=@id	UPDATE sales_orders SET ... WHERE id=@id AND factory_id=@factoryId	多一个 AND 条件
前端页面	无需关注工厂	无需关注工厂（中间件自动处理）	无差异

10.6 结论与建议

结论：综合考虑开发效率、代码质量、数据安全、可维护性和总体工期，**强烈推荐采用路线B的改良版“骨架先行，渐进填充”策略**。前期投入 3-5 天搭建多工厂骨架，后续所有功能开发体验与单工厂几乎无差异，但架构基础正确，避免了“先建后拆”的巨大返工成本（预估节省 15-25 人天）。

建议的具体执行顺序：

- 1. 第1步 (Day 1-2)：创建 factory 表 + users 表加 default_factory_id + 编写种子数据脚本
- 2. 第2步 (Day 2-3)：实现 factoryScope 中间件 + factoryWhere 工具函数 + 注册到 Express 中间件链
- 3. 第3步 (Day 3-4)：编写一个示例模块（如仓库模块），验证隔离效果
- 4. 第4步 (Day 4-5)：制定开发规范文档，团队内部培训
- 5. 第5步 (Day 5+)：按正常节奏逐模块开发业务功能，框架已透明化

十一、用户登录与工厂访问方案

核心问题：在多工厂架构下，普通用户如何登录到自己所在的工厂？系统的登录流程是否需要改造？用户能否在工厂之间切换？

11.1 当前系统登录流程

当前登录流程为经典的 JWT 无状态认证模式：

登录页 → POST /api/v1/auth/login {username, password}

→ 验证用户名密码 → 检查账户锁定/禁用状态

→ JWT 签发 {id, username, role}

→ 返回 {token, user, permissions}

→ 前端存 localStorage → 进入系统

关键特征：

- JWT Token 中仅含 {id, username, role}，**没有工厂信息**
- auth.middleware.ts 解析 Token 后设置 req.user = {id, username, real_name, role}
- 前端登录页为简单的用户名+密码表单，无工厂选择

11.2 三种可选方案对比

方案	方案1：后台绑定 (静默分配)	方案2：登录页选择 (显式选择)	方案3：登录后切换 (顶部切换器)
操作方式	管理员在后台将用户绑定到工厂；用户登录时无需选择，系统自动识别	登录页增加工厂下拉框；用户先选工厂再输入用户名密码	用户名密码登录后，顶部栏显示工厂切换器，可随时切换
登录页是否改动	✅ 无需改动	❌ 需要增加工厂下拉框	✅ 无需改动
一个账号多工厂	❌ 不支持（一个账号仅绑定一个工厂）	✅ 支持（通过权限表控制）	✅ 支持（通过权限表控制）

安全性	✅ 最高 — 前端无法篡改工厂归属	⚠️ 中等 — 需后端校验用户是否有该工厂权限	⚠️ 中等 — 切换工厂需刷新Token，后端校验
实施复杂度	✅ 简单 — JWT 加字段 + 用户表单加下拉	⚠️ 中等 — 登录页 + 权限表 + 校验逻辑	⚠️ 中等 — 切换器组件 + Token 刷新 + 权限表
适用场景	操作员（一个员工仅在一个工厂工作）	用户知道自己要去哪个工厂	管理人员需要跨工厂查看

11.3 推荐方案：方案1 + 方案3 组合

结合两个工厂的实际场景，推荐采用 **以方案1为主（默认绑定），辅以方案3（工厂切换器）** 的混合策略：

Molding Powder MOM

当前工厂：宁国一厂 ▼

← 仅多工厂权限用户可见

普通操作员：管理员在后台将其绑定到固定工厂

→ 登录后自动进入绑定工厂，无需选择，也看不到切换器

管理人员/总部：管理员赋予多工厂权限

→ 登录后默认进入主工厂，可点击顶部切换器换厂

11.4 数据库层改动

11.4.1 users 表增加工厂字段

```
-- users 表新增字段
ALTER TABLE users ADD default_factory_id INT NULL;
ALTER TABLE users ADD CONSTRAINT fk_user_factory
    FOREIGN KEY (default_factory_id) REFERENCES factory(id);

-- 已有用户批量归入默认工厂
UPDATE users SET default_factory_id = 1 WHERE default_factory_id IS NULL;
```

11.4.2 用户-工厂多对多权限表（用于方案3的切换器）

```
-- 用户可访问的工厂列表（仅需跨工厂的用户才需要配置）
CREATE TABLE user_factory_access (
    id INT IDENTITY(1,1) PRIMARY KEY,
    user_id INT NOT NULL,
    factory_id INT NOT NULL,
    is_default BIT DEFAULT 0,           -- 是否默认工厂
    created_at DATETIME DEFAULT GETDATE(),
    CONSTRAINT fk_ufa_user FOREIGN KEY (user_id) REFERENCES users(id),
```

```

CONSTRAINT fk_ufa_factory FOREIGN KEY (factory_id) REFERENCES factory(id),
CONSTRAINT uq_user_factory UNIQUE (user_id, factory_id)
);

```

11.5 后端改动

11.5.1 登录接口 — JWT 注入 factory_id

```

// auth.controller.ts → login 函数
const token = generateToken({
  id: user.id,
  username: user.username,
  role: user.role,
  factory_id: user.default_factory_id // ← 新增：注入工厂归属
});

res.json(success({
  token,
  user: {
    id: user.id,
    username: user.username,
    real_name: user.real_name,
    role: user.role,
    factory_id: user.default_factory_id, // ← 新增：返回给前端显示
    ...
  },
  permissions: permInfo
})));

```

11.5.2 Token 刷新也带上 factory_id

```

// auth.controller.ts → refreshToken 函数
const newToken = generateToken({
  id: user.id,
  username: user.username,
  role: user.role,
  factory_id: user.default_factory_id // ← 同样注入
});

```

11.5.3 auth.middleware.ts — 解析 factory_id

```

// auth.middleware.ts → authenticate 函数
const decoded = verifyToken(token);
// decoded 现在包含: {id, username, role, factory_id, iat, exp}

const user = await User.findByPk(decoded.id, {
  attributes: { exclude: ['password'] }
});

req.user = {
  id: user.id,
  username: user.username,

```

```

    real_name: (user as any).real_name || '',
    role: user.role,
    factory_id: decoded.factory_id || user.default_factory_id // ← 新增
  };

```

11.5.4 工厂切换接口（用于方案3顶部切换器）

```

// 新增接口: POST /api/v1/auth/switch-factory
// 用户点击顶部切换器时调用, 生成新 Token
const switchFactory = async (req, res) => {
  const { factory_id } = req.body;

  // 验证用户是否有权访问目标工厂
  const hasAccess = await checkUserFactoryAccess(req.user.id, factory_id);
  if (!hasAccess) {
    return res.status(403).json(error('无权限访问该工厂'));
  }

  // 生成新 Token, 包含目标 factory_id
  const newToken = generateToken({
    id: req.user.id,
    username: req.user.username,
    role: req.user.role,
    factory_id: factory_id
  });

  res.json(success({ token: newToken }, '工厂切换成功'));
};

```

11.6 前端改动

11.6.1 登录页 — 无需改动

登录页保持原有的用户名+密码表单，**不增加工厂选择**。用户的工厂归属由后台管理员配置，登录时自动确定。

11.6.2 顶部栏 — 工厂切换器（仅多工厂权限用户可见）

```

<!-- 顶部栏右侧, 仅当用户有多个可访问工厂时显示 -->
<a-dropdown v-if="userStore.accessibleFactories.length > 1">
  <a-button type="text">
    <ShopOutlined />
    当前工厂: {{ userStore.currentFactory.name }}
    <DownOutlined />
  </a-button>
  <template #overlay>
    <a-menu @click="handleSwitchFactory">
      <a-menu-item
        v-for="f in userStore.accessibleFactories"
        :key="f.id"
      >
        {{ f.factory_name }}
        <a-tag v-if="f.id === userStore.currentFactory.id">当前</a-tag>
      </a-menu-item>
    </a-menu>
  </template>
</a-dropdown>

```

```
</a-menu>
</template>
</a-dropdown>
```

11.6.3 用户管理页面 — 新增工厂选择字段

在创建/编辑用户的表单中增加"所属工厂"下拉选择：

创建用户

用户名：

姓名：

角色：

普通员工

▼

所属工厂：

宁国一厂

▼

← 新增

密码：

保存

取消

11.7 用户体验完整流程

用户类型	登录流程	进入系统后
工厂A操作员 (绑定工厂A)	1. 打开登录页 2. 输入用户名+密码 3. 点击登录 4. 系统自动识别其归属工厂A	· 顶部栏无工厂切换器 · 所有数据自动限定为工厂A · 创建单据自动归属工厂A
工厂B操作员 (绑定工厂B)	1. 打开登录页 2. 输入用户名+密码 3. 点击登录 4. 系统自动识别其归属工厂B	· 顶部栏无工厂切换器 · 所有数据自动限定为工厂B · 创建单据自动归属工厂B
跨工厂管理人员 (多工厂权限)	1. 打开登录页 2. 输入用户名+密码 3. 点击登录 4. 系统进入其默认工厂（如工厂A）	· 顶部栏显示 "当前工厂：宁国一厂 ▼" · 点击可切换到工厂B · 切换后页面刷新，数据变为目标工厂 · 可随时在两个工厂间切换
总部 Admin (headquarters 角色)	1. 打开登录页 2. 输入用户名+密码 3. 点击登录 4. 系统不限定工厂	· 顶部栏显示 "总部视图" · 可查看/切换所有工厂数据 · 可访问总部汇总表 · 可管理工厂基础配置

11.8 安全性保障

- **factory_id 不可从前端传入：**JWT 中的 factory_id 由后端在登录时根据用户绑定关系生成，前端无法篡改
- **工厂切换需后端校验：**切换工厂接口验证 user_factory_access 表，防止越权访问
- **Token 即时刷新：**切换工厂后立即签发新 Token，旧 Token 中的 factory_id 失效

- **双重保障**：factoryScope 中间件 + SQL 层 WHERE 条件，即使 JWT 被篡改，数据库层也能兜底

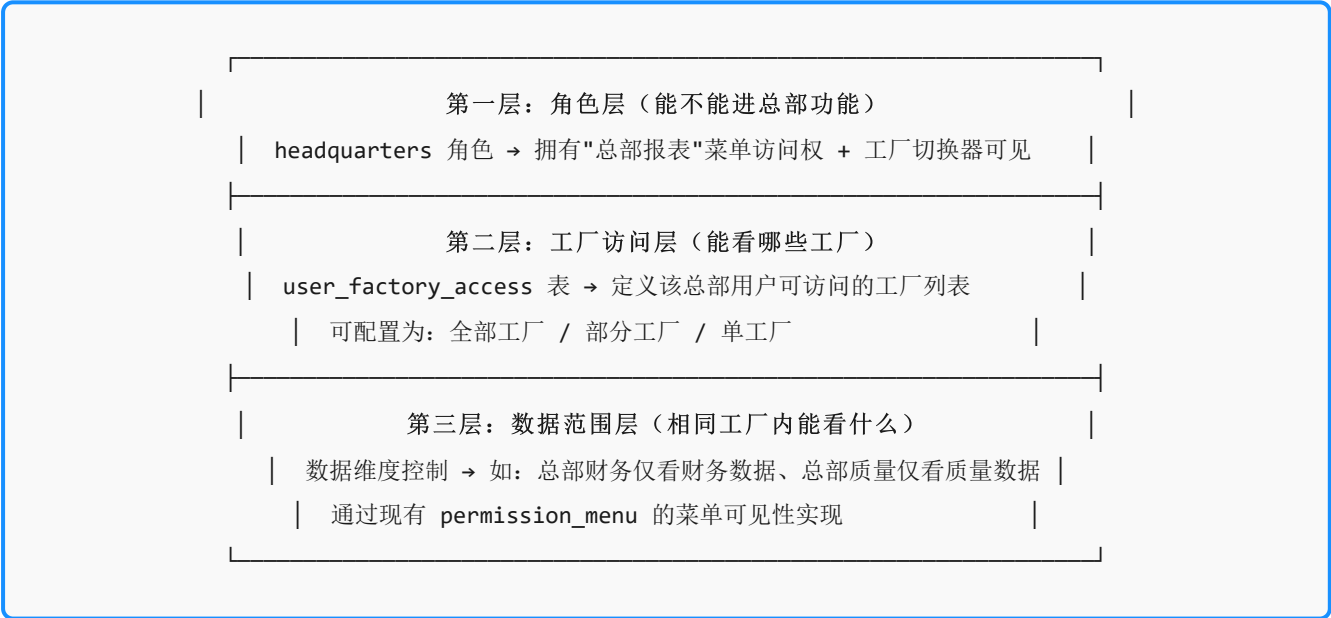
小结：对于普通操作员，登录体验与单工厂完全一致（用户名+密码即可）；对于需要跨工厂的管理人员，登录后在顶部栏一键切换。整个方案**前端改动极小**（登录页不动，用户管理加一个下拉框，顶部栏加一个切换器），**安全性由后端全链路保障**。

十二、总部权限精细化管控方案

核心问题：总部人员如何实现“能看到多个工厂所有数据、自由切换工厂视图、访问集团汇总报表”？这需要在角色设计、工厂访问权限、报表权限三个层面建立精细化的管控模型。

12.1 权限管控三层模型

总部权限不是“一个角色打天下”，而是遵循**三层递进模型**：



12.2 总部角色体系设计

摒弃“一个 headquarters 角色管全部”的粗放方式，建立**总部角色梯队**：

角色编码	角色名称	能力范围	适用人员
headquarters_admin	总部超级管理员	· 查看所有工厂全部数据 · 访问所有总部汇总报表 · 管理工厂基础配置 · 管理用户和权限分配	集团IT负责人、系统管理员
headquarters_manager	总部管理层	· 查看所有工厂全部业务数据 · 访问所有总部汇总报表	总经理、集团运营总监

		· 可切换工厂视图 · 不可修改基础配置	
headquarters_finance	总部财务	· 查看所有工厂的财务相关数据 · 访问集团财务汇总报表 · 可查看各工厂财务明细 · 不可查看生产/质量数据	集团财务总监、总部会计
headquarters_quality	总部质量	· 查看所有工厂的质量数据 · 访问集团质量汇总报表 · 可查看各工厂质检明细 · 不可查看财务/成本数据	集团质量总监
headquarters_sales	总部销售	· 查看所有工厂的销售数据 · 访问集团销售汇总报表 · 跨工厂客户分析 · 不可查看生产成本数据	集团销售总监

12.3 工厂访问权限表设计

```
-- 用户工厂访问权限表（精细化版本）
CREATE TABLE user_factory_access (
  id INT IDENTITY(1,1) PRIMARY KEY,
  user_id INT NOT NULL,
  factory_id INT NOT NULL,
  access_level NVARCHAR(20) DEFAULT 'read', -- 'read'仅查看 | 'write'可操作 | 'admin'可配置
  is_default BIT DEFAULT 0,                -- 是否默认登录工厂
  created_by INT,
  created_at DATETIME DEFAULT GETDATE(),
  CONSTRAINT fk_ufa_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_ufa_factory FOREIGN KEY (factory_id) REFERENCES factory(id),
  CONSTRAINT uq_user_factory UNIQUE (user_id, factory_id)
);

-- 示例：总部管理层可以查看两个工厂的全部数据
INSERT INTO user_factory_access (user_id, factory_id, access_level, is_default) VALUES
  (10, 1, 'write', 1),    -- 总部经理 → 工厂A（默认登录，可操作）
  (10, 2, 'write', 0);   -- 总部经理 → 工厂B（可切换，可操作）

-- 示例：总部财务可以查看两个工厂但不可操作
INSERT INTO user_factory_access (user_id, factory_id, access_level, is_default) VALUES
  (11, 1, 'read', 1),     -- 总部财务 → 工厂A（默认登录，只读）
  (11, 2, 'read', 0);    -- 总部财务 → 工厂B（可切换，只读）
```

12.4 factoryScope 中间件增强 — 支持三种视图模式

```
// 增强版 factoryScope 中间件
function factoryScope(req, res, next) {
  const user = req.user;

  // 1. 总部角色 + 选择了"全部工厂"视图
  if (isHQRole(user.role) && req.headers['x-view-mode'] === 'all') {
```

```

    req.factoryScope = {
      mode: 'all',          // 全工厂视图
      filter: false,        // 不过滤 factory_id
      accessibleFactories: [] // 所有有权限的工厂
    };
    return next();
  }

  // 2. 总部角色 + 选择了特定工厂视图
  if (isHQRole(user.role) && req.headers['x-factory-id']) {
    const targetFactoryId = parseInt(req.headers['x-factory-id']);
    // 验证该用户是否有此工厂的访问权限
    if (!hasFactoryAccess(user.id, targetFactoryId)) {
      return res.status(403).json({
        success: false,
        message: '无权限访问该工厂'
      });
    }
    req.factoryScope = {
      mode: 'single',
      factory_id: targetFactoryId,
      filter: true
    };
    return next();
  }

  // 3. 普通用户 / 总部用户默认进入其绑定工厂
  const factoryId = user.default_factory_id ||
    (user.accessibleFactories && user.accessibleFactories[0]);
  req.factoryScope = {
    mode: 'single',
    factory_id: factoryId,
    filter: true
  };
  next();
}

// 判断是否为总部角色
function isHQRole(role) {
  return ['headquarters_admin', 'headquarters_manager',
    'headquarters_finance', 'headquarters_quality',
    'headquarters_sales'].includes(role);
}

// 验证用户是否有某工厂的访问权限
async function hasFactoryAccess(userId, factoryId) {
  const access = await sequelize.query(
    'SELECT 1 FROM user_factory_access WHERE user_id = @uid AND factory_id = @fid',
    { replacements: { uid: userId, fid: factoryId }, type: QueryTypes.SELECT }
  );
  return access.length > 0;
}

```

12.5 顶部栏 — 总部用户专属视图切换器


```
<!-- 总部用户可见的增强版视图切换器 -->
<template>
  <div class="factory-view-switcher" v-if="isHQUser">
    <a-radio-group
      :value="currentViewMode"
      @change="handleViewModeChange"
      button-style="solid"
      size="small"
    >
      <a-radio-button value="all">
        <GlobalOutlined /> 全部工厂
      </a-radio-button>
      <a-radio-button
        v-for="f in accessibleFactories"
        :key="f.id"
        :value="'factory_' + f.id"
      >
        {{ f.factory_short }}
      </a-radio-button>
    </a-radio-group>
  </div>
</template>

<script setup>
const currentViewMode = ref('all'); // 默认全部工厂视图

const handleViewModeChange = (e) => {
  if (e.target.value === 'all') {
    // 切换到全部工厂模式 → 所有 API 请求头携带 x-view-mode: all
    apiClient.defaults.headers['x-view-mode'] = 'all';
    delete apiClient.defaults.headers['x-factory-id'];
  } else {
    // 切换到单工厂模式 → 携带 x-factory-id
    const factoryId = e.target.value.replace('factory_', '');
    apiClient.defaults.headers['x-factory-id'] = factoryId;
    delete apiClient.defaults.headers['x-view-mode'];
  }
  currentViewMode.value = e.target.value;
  // 触发页面数据重新加载
  router.go(0); // 或使用 provide/inject 触发全局刷新
};
</script>
```

12.6 三种视图模式的完整交互流程

视图模式	顶部栏显示	API 请求头	查询结果
全部工厂	[● 全部工厂] [工厂A] [工厂B] 默认选中"全部工厂"	x-view-mode: all 不携带 factory_id	列表页：每条数据显示所属工厂 汇总页：按工厂分组统计 报表：跨工厂聚合
单工厂A	[全部工厂] [● 工厂A] [工厂B] 选中"工厂A"	x-factory-id: 1 携带 factory_id=1	完全等同于工厂A操作员视角 数据隔离，仅显示工厂A数据

单工厂B	[全部工厂] [工厂A] [● 工厂B] 选中"工厂B"	x-factory-id: 2 携带 factory_id=2	完全等同于工厂B操作员视角 数据隔离，仅显示工厂B数据
------	---------------------------------	------------------------------------	--------------------------------

12.7 总部汇总报表的权限控制

集团汇总报表独立于日常业务操作，通过**菜单权限 + 角色白名单**双控机制：

```
// 总部报表路由 - 独立权限验证
// server/src/modules/headquarters/headquarters.routes.ts

router.use(authenticate); // 先验证登录
router.use(authorizeHQ);  // 再验证总部角色

// 总部权限中间件
function authorizeHQ(req, res, next) {
  const allowedRoles = [
    'headquarters_admin',
    'headquarters_manager',
    'headquarters_finance',
    'headquarters_quality',
    'headquarters_sales'
  ];

  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({
      success: false,
      message: '仅总部人员可访问汇总报表'
    });
  }
  next();
}

// 总部报表路由（不使用 factoryScope 中间件）
router.get('/sales-summary',      authorizeHQ, salesSummaryController.list);
router.get('/production-summary', authorizeHQ, productionSummaryController.list);
router.get('/purchase-summary',   authorizeHQ, purchaseSummaryController.list);
router.get('/inventory-summary',  authorizeHQ, inventorySummaryController.list);
router.get('/finance-summary',    authorizeHQ, financeSummaryController.list);
router.get('/quality-summary',    authorizeHQ, qualitySummaryController.list);
```

报表级权限细化

角色	销售汇总	生产汇总	采购汇总	库存汇总	财务汇总	质量汇总
headquarters_admin	✓	✓	✓	✓	✓	✓
headquarters_manager	✓	✓	✓	✓	✓	✓
headquarters_finance	✓	✗	✓	✓	✓	✗
headquarters_quality	✗	✗	✗	✗	✗	✓
headquarters_sales	✓	✗	✗	✓	✗	✗

12.8 完整权限配置流程（管理员操作）



12.9 总部用户的完整体验流程

场景	操作	系统表现
登录	用户名+密码 → 登录	· JWT 注入角色+可访问工厂列表 · 默认进入"全部工厂"视图 · 顶部栏显示 [●全部工厂] [工厂A] [工厂B]
查看销售订单 (全部工厂模式)	点击"销售订单"菜单	· 列表中每条订单显示"工厂"列 · 可筛选/排序工厂维度 · 数据来自两个工厂，不隔离
切换到单工厂A	点击顶部 [工厂A] 按钮	· API 请求自动携带 x-factory-id:1 · 页面刷新，仅显示工厂A数据 · 与工厂A操作员看到完全一致
查看集团销售 汇总	总部报表 → 集团销售汇总	· 跨工厂聚合统计 · 图表按工厂分色对比 · KPI 卡片显示集团总量
钻取明细	在汇总报表中点击某工厂的柱状图	· 自动切换到该工厂单视图 · 展示该工厂的订单明细列表 · 支持进一步筛选和导出

12.10 安全性约束总结

安全层级	机制	防护目标
认证层	JWT 解析 → 确定用户身份和角色	防止未登录访问
角色层	authorizeHq 中间件 → 只放行总部角色	防止普通用户访问总部报表
工厂访问层	user_factory_access 表 → 验证每个工厂的访问权限	防止越权查看未授权的工厂

数据范围层	permission_menu → 控制菜单可见性	防止总部财务查看生产数据等横向越权
操作层	access_level (read/write/admin)	防止只读用户执行写操作